

Assignment 12: Neural Network and Deep Learning Basics

Classifying clothing with a feedforward neural network

A 109,386-parameter network reaches 88.65% on Fashion MNIST, and every remaining error has the same cause.

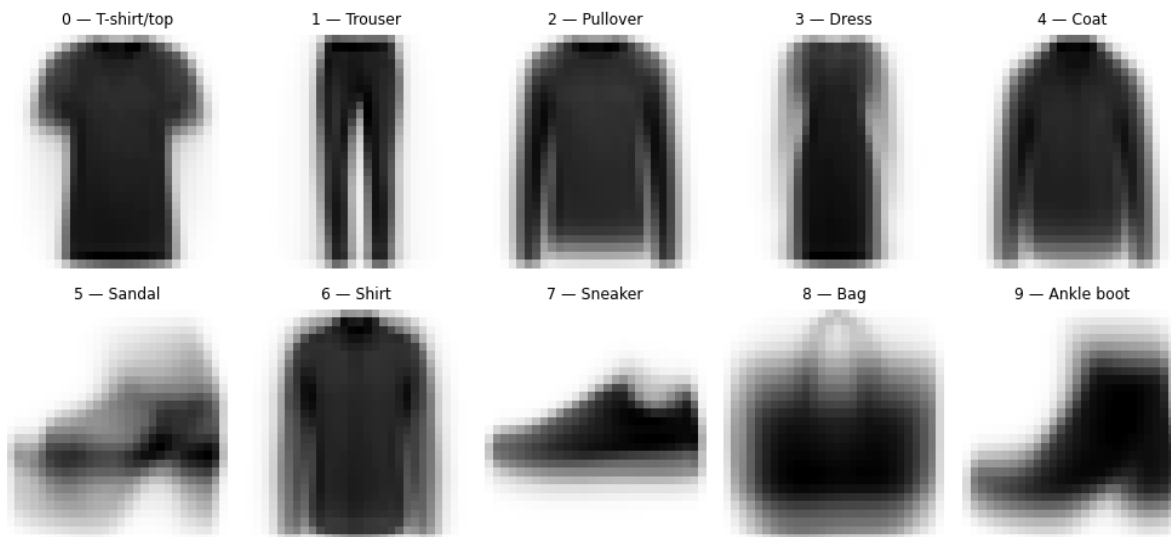
Ayleen Santander

Assignment 12 — Neural Networks and Deep Learning Basics · TensorFlow 2.20.0 / Keras · Fashion MNIST

Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
0.8865	0.8878	0.8865	0.8866

Held-out test set, 10,000 images the model never saw during training.

Average image per class (blurrier = more variety)



The problem in one image. Each panel averages all 6,000 training images of a class. Trouser, Sandal and Bag stay sharp, meaning the class has a consistent shape. T-shirt, Pullover, Coat and Shirt average into nearly the same torso silhouette. Those four classes account for most of the model's errors.

1. Introduction

The dataset

Fashion MNIST is a set of 70,000 grayscale images of clothing items published by Zalando Research as a drop-in replacement for the original handwritten-digit MNIST. Each image is 28×28 pixels with values from 0 to 255, and each belongs to exactly one of ten categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot.

The split is 60,000 training images and 10,000 test images, and the classes are perfectly balanced: 6,000 examples per class in training and 1,000 per class in test. That balance matters for how the results get read. With no majority class to lean on, plain accuracy is a trustworthy headline metric here, which is not true of most real classification problems.

The data was loaded twice for verification. First through `keras.datasets.fashion_mnist`, then directly from the four raw IDX files in the Zalando GitHub repository using a custom loader. Both paths returned identical shapes, confirming the download was intact.



Six random training examples per class. Items are centred and shot on a plain background, but pose, proportion and texture vary widely inside each class. Footwear and bags are visually distinct; the upper-body garments overlap heavily.

The model

The main model is a feedforward neural network, also called a multilayer perceptron. It takes the flattened 784-pixel vector as input and passes it through two fully connected hidden layers with ReLU activation, each followed by a dropout layer, before a 10-unit softmax output that produces one probability per class.

Written as a layer stack: $784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(10, \text{softmax})$, for a total of 109,386 trainable parameters.

ReLU is used in the hidden layers because it keeps gradients from vanishing in deeper stacks and is cheap to compute. Softmax is used on the output because the ten classes are mutually exclusive, and softmax forces the ten outputs to sum to 1, so each one can be read as a probability. Dropout randomly switches off 20% of the units in a layer during each training step, which stops the network from depending too heavily on any single feature.

A convolutional network was also built for comparison in the improvement experiments, and a logistic regression served as the non-neural baseline.

2. Methodology

2.1 Preprocessing

Three transformations were applied, each one required by a different model in the study.

Normalization. Pixel values were cast to float32 and divided by 255, moving the range from 0–255 to 0–1. Neural networks train on gradients, and gradients computed from inputs in the hundreds produce weight updates that overshoot. Scaling to the unit interval keeps those updates stable and lets the optimizer converge in far fewer epochs.

Reshaping. Three shapes were produced from the same source arrays. (60000, 28, 28) feeds the feedforward network, which flattens internally. (60000, 784) feeds scikit-learn, which only accepts 2D input. (60000, 28, 28, 1) adds the explicit channel dimension that Keras convolutional layers and ImageDataGenerator require.

Label encoding. Labels arrive as integers 0 through 9. They were one-hot encoded with `to_categorical` into 10-element binary vectors so the model could be compiled with `categorical_crossentropy`, as the assignment specifies. Keras also offers `sparse_categorical_crossentropy`, which is the identical loss function and simply accepts the integer labels without the encoding step. The one-hot version was used so the compile call matches the requirement literally.

No data leakage risk exists in this preprocessing. Dividing by 255 is a fixed constant rather than a statistic estimated from the data, so nothing about the test set influences the transformation applied to it.

2.2 Architecture details

Layer	Output shape	Parameters
Flatten	(None, 784)	0
Dense — ReLU	(None, 128)	100,480

Layer	Output shape	Parameters
Dropout (0.2)	(None, 128)	0
Dense — ReLU	(None, 64)	8,256
Dropout (0.2)	(None, 64)	0
Dense — softmax	(None, 10)	650
Total		109,386

Table 1. Feedforward network summary. The first dense layer holds 92% of the parameters, since it connects all 784 inputs to 128 units.

Activations were verified programmatically after construction rather than assumed from the code, by looping over `model.layers` and printing each layer's activation function. The output confirmed ReLU on both hidden layers and softmax on the output.

2.3 Training

The model was compiled with the Adam optimizer, `categorical_crossentropy` loss and accuracy as the tracked metric. Adam was chosen because it adapts the learning rate per parameter, which usually reaches a good solution in fewer epochs than plain stochastic gradient descent and needs almost no manual tuning.

Training ran with a batch size of 64 and `validation_split=0.1`, which holds out the last 6,000 training images as a validation set. That leaves 54,000 images for weight updates, or 844 batches per epoch. The test set was never touched during training.

Note

The final model was fitted across three successive `model.fit()` calls on the same model object: 5 epochs, then 5 more, then 15. Keras does not reset weights between calls, so training continued from the existing weights each time and the network saw roughly 25 epochs in total. This is why the accuracy curve in Figure 3 begins near 0.89 rather than near 0.10. Every number reported here is correct for the model as trained, but a clean rerun should use a single `fit()` call so the epoch count on the x-axis matches the actual training history.

2.4 Evaluation metrics

Four metrics were computed on the test set, all macro-averaged so that every class contributes equally regardless of how easy it is.

- **Accuracy** is the share of all test images classified correctly. It is a fair summary here only because the classes are balanced.
- **Precision** answers, of everything the model labelled as class X, what fraction actually was class X. Low precision means the class is over-predicted.
- **Recall** answers, of all the real class X images, what fraction the model found. Low recall means the class is being missed.
- **F1-score** is the harmonic mean of precision and recall, which penalises a model that buys one at the expense of the other.

A confusion matrix was produced alongside these, in both raw-count and row-normalized form. The four summary numbers say how well the model performs; only the matrix says which specific pairs of classes it mixes up, which is the part that drives every improvement decision later in the report.

3. Results

3.1 Baseline

Before training any network, a logistic regression inside a Pipeline with StandardScaler was fitted on a random 10,000-image subset to establish a floor. It reached **96.62% on its own training data and 79.95% on the test set**. That 16-point gap is textbook overfitting: with 784 raw pixel features and a linear decision boundary, the model memorises the subset it saw instead of learning shape.

The baseline sets the bar the neural network has to beat, and it also previews the outcome. Its weakest classes were Shirt (F1 0.53), Pullover (0.67) and Coat (0.69), exactly the group that stays hardest for every model tested afterwards.

3.2 Training and validation curves

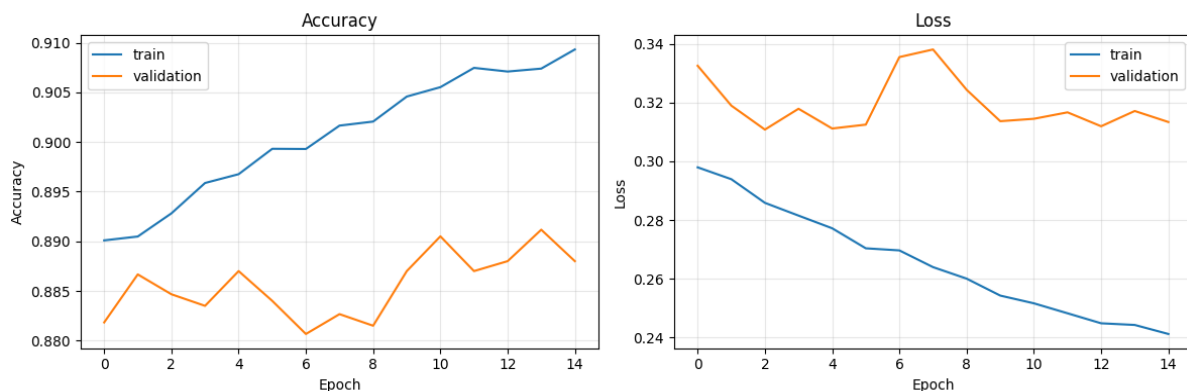


Figure 3. Accuracy and loss over the final 15 epochs. Training accuracy climbs from 0.890 to 0.909 and training loss falls from 0.298 to 0.241. Validation accuracy stays flat between 0.880 and 0.891, and validation loss never drops below its starting value. The curves separate after roughly epoch 5.

The two curves telling different stories is the main finding here. Training loss keeps falling because the network keeps fitting the training set more closely. Validation loss stops improving, which means the extra fitting is not producing anything that generalises. The model is memorising rather than learning past that point.

The gap is mild. Roughly two points separate training and validation accuracy, compared with sixteen for the logistic baseline, which shows the dropout layers are doing their job. But the flat validation curve is the practical signal that more epochs would not help. Training beyond epoch 5 of this run bought nothing, and EarlyStopping with patience=3 monitoring val_loss would have stopped there automatically.

3.3 Test set performance

The model reached **88.65% accuracy** on the 10,000 held-out test images, with macro precision 0.8878, macro recall 0.8865 and macro F1 0.8866. The three averages sitting within 0.0013 of each other confirms the model is not trading precision for recall on any class in a way that skews the overall picture.

Class	Precision	Recall	F1	Support
Trouser	0.9908	0.9690	0.9798	1000
Bag	0.9692	0.9740	0.9716	1000
Sandal	0.9699	0.9680	0.9690	1000
Ankle boot	0.9516	0.9630	0.9573	1000
Sneaker	0.9527	0.9460	0.9493	1000
Dress	0.8553	0.9100	0.8818	1000
T-shirt/top	0.8596	0.8020	0.8298	1000
Coat	0.7796	0.8490	0.8128	1000
Pullover	0.8471	0.7590	0.8006	1000
Shirt	0.7025	0.7250	0.7136	1000

Table 2. Per-class results, sorted by F1. The five shaded classes score above 0.94. The four in amber, all upper-body garments, score below 0.83. The spread between best and worst is 27 points.

Splitting the results this way is more informative than the single accuracy figure. Five classes are essentially solved. Trouser, Bag, Sandal, Ankle boot and Sneaker all clear 0.94 F1 because each has a distinctive outline that survives being flattened into a pixel vector. The remaining four are upper-body garments that share a torso-plus-sleeves silhouette, and they carry almost the entire error budget.

3.4 Confusion matrix

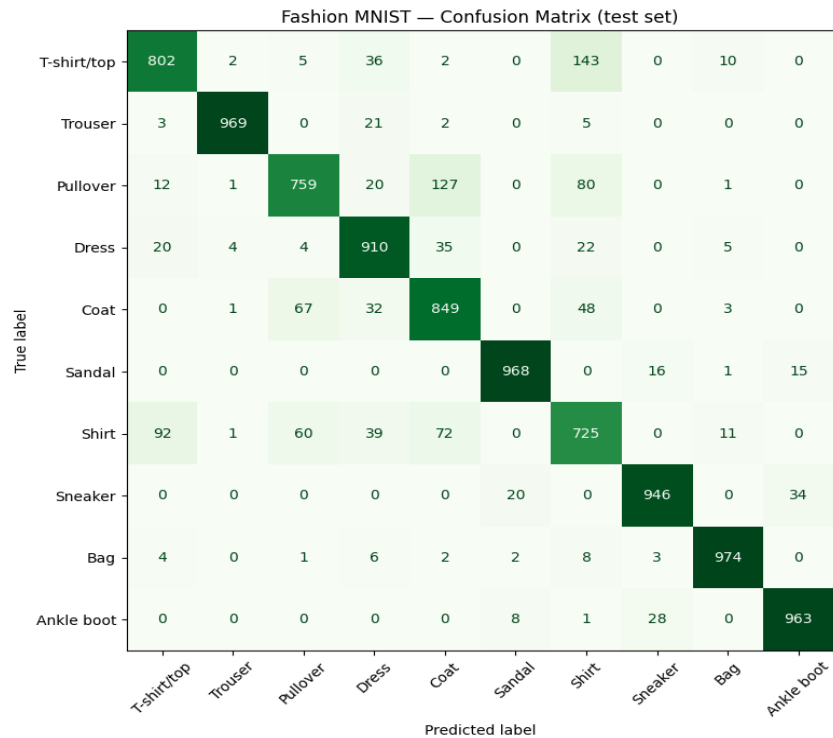


Figure 4. Confusion matrix, raw counts. Rows are true labels, columns are predictions. Correct classifications sit on the diagonal.

The errors are not scattered. They cluster in a single 4×4 block covering T-shirt, Pullover, Coat and Shirt:

- 143 T-shirts predicted as Shirt, the single largest error cell in the matrix
- 127 Pullovers predicted as Coat
- Shirt scatters in three directions: 92 to T-shirt, 72 to Coat, 60 to Pullover
- 67 Coats predicted as Pullover and 48 as Shirt

Adding those up, roughly 70% of all test-set errors fall inside that block. The only meaningful confusion outside it is footwear, where 34 Sneakers were read as Ankle boot and 28 went the other way. Everything else is noise at the single-digit level.

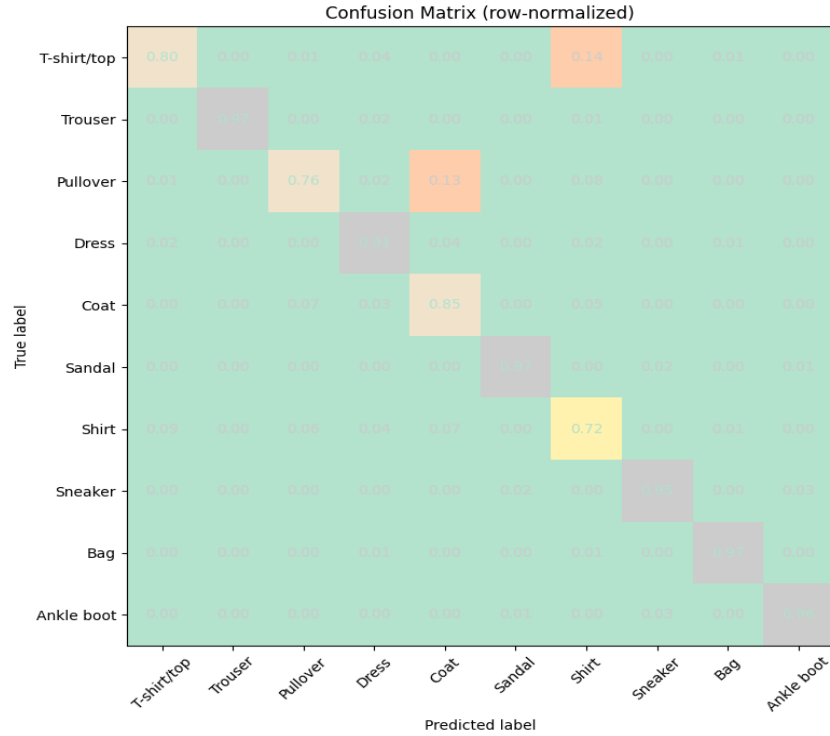


Figure 5. The same matrix, row-normalized. Each row sums to 1, so the diagonal reads directly as per-class recall. This view makes the Shirt row, at 0.73 correct, comparable at a glance to the Trouser row at 0.97.

This connects directly back to the average-image figure at the top of the report. Those four classes average into near-identical silhouettes because they genuinely are near-identical at 28×28 grayscale. What separates a shirt from a t-shirt is collar detail, button placket and cuff texture, and those features occupy a handful of pixels that largely vanish at this resolution. A human annotator shown the same images in isolation would also hesitate. The ceiling here is partly in the data, not only in the model.

3.5 Architecture and optimizer experiments

Six configurations were trained for 15 epochs each under identical conditions to isolate the effect of depth, width and optimizer choice.

Architecture	Optimizer	Parameters	Best val acc.	Test acc.
2 layers [512, 256]	Adam	535,818	0.8967	0.8924
3 layers [256, 128, 64]	Adam	242,762	0.8877	0.8834
2 layers [128, 64]	Adam	109,386	0.8872	0.8820
1 layer [128]	Adam	101,770	0.8902	0.8784
2 layers [128, 64]	RMSprop	109,386	0.8878	0.8737
2 layers [128, 64]	SGD	109,386	0.8590	0.8556

Table 3. Six configurations, 15 epochs each, sorted by test accuracy. The bottom three rows share an identical architecture and differ only in optimizer.

Three things come out of this.

Width helped more than depth. The widest network, [512, 256], was the strongest at 0.8924. But going from one hidden layer to three moved test accuracy by half a point while adding 141,000 parameters. For a 28×28 input, extra depth in a fully connected network has very little left to learn.

Optimizer choice mattered more than architecture. The last three rows are the same network trained three different ways. Swapping Adam for SGD cost 2.6 points, a bigger swing than any architectural change tested. Adam's per-parameter adaptive learning rate converges much faster at a fixed 15-epoch budget. SGD is not a worse optimizer in principle, but it needs learning-rate tuning and more epochs to compete.

Returns flattened quickly. Going from 101,770 to 535,818 parameters, a factor of five, bought 1.4 points. That is the signature of an architecture running into its own limits rather than a lack of capacity.

3.6 Data augmentation

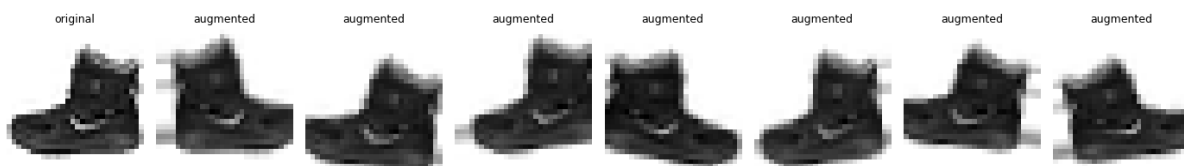


Figure 6. One image, seven augmented variants. Rotation up to 10° , width and height shifts up to 10%, zoom up to 10%, and horizontal flipping, with exposed pixels filled by nearest-neighbour.

Augmentation only makes sense as a test on a convolutional network. Flattening an image discards the spatial structure that rotations and shifts are meant to teach the model to tolerate, so a dense network gains little from it. A CNN with two convolutional blocks and 422,026 parameters was therefore built and trained twice on the identical train/validation split, once with augmentation and once without.

Model	Test accuracy
CNN, no augmentation	0.9109
CNN, with augmentation	0.8862
Difference	-0.0247

Table 4. Augmentation comparison, 5 epochs each, same split.

Augmentation made the model worse by 2.5 points, and the reason is the epoch budget rather than a flaw in the technique. Augmentation deliberately makes the training problem harder, showing the network a different distorted version of each image every epoch. A model needs considerably more epochs to reach the same fit on that harder problem, and its payoff only appears afterwards, as better generalisation once both models have converged. Comparing them at 5 epochs measures how fast each one converges, not how well each one generalises. The honest conclusion is that this test was cut short, not that augmentation hurts.

Two of the transformations are also questionable for this dataset specifically. Horizontal flipping assumes clothing is left-right symmetric, which is mostly true for these silhouettes but not for asymmetric items. And rotation plus shift can push the thin detail near a garment's edge outside the 28-pixel frame, destroying precisely the collar and cuff information the model most needs to separate Shirt from T-shirt.

The clean finding is elsewhere in the same table. The unaugmented CNN at 0.9109 beat the best feedforward network at 0.8924 while using 5 epochs instead of 15. Convolution is the right inductive bias for image data

because it looks at local pixel neighbourhoods and reuses the same filters across the whole image, which is exactly the structure flattening throws away.

3.7 Single-image prediction

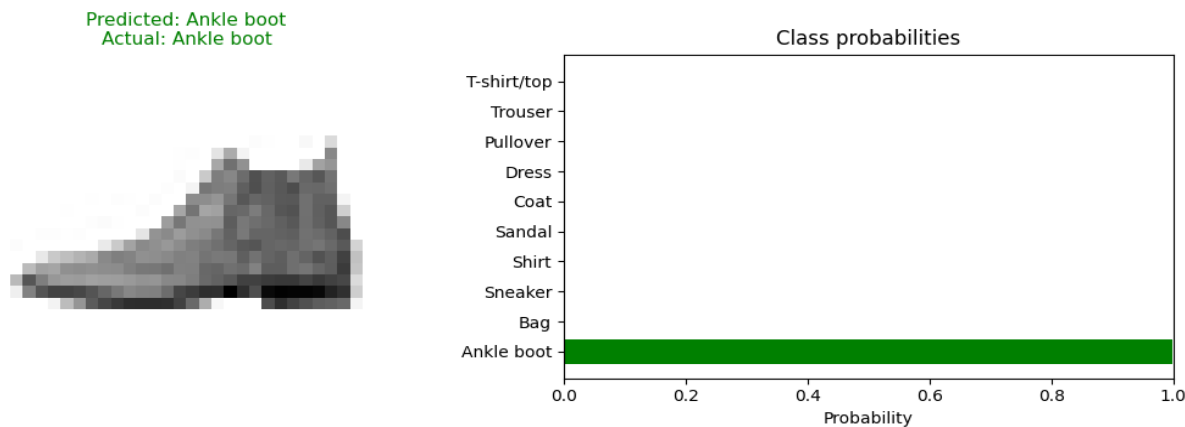


Figure 7. Test image 0, predicted correctly as Ankle boot at 99.97% confidence. The softmax output puts essentially all probability mass on one class.

Inspecting the full probability vector rather than the single predicted label is what makes a model deployable. Confidence this high signals a class the model finds unambiguous. On a Shirt image the same chart typically spreads across three or four bars, and that spread is a usable runtime signal: when the top probability falls below a threshold, the application can offer several options instead of asserting one.

4. Discussion

A realistic deployment

The natural application is a peer-to-peer resale marketplace such as Poshmark or Depop, where individual sellers create their own listings. Sellers routinely skip the category dropdown, choose “Other”, or mislabel a pullover as a coat. Miscategorised items never surface in filtered searches, so they do not sell, and sellers who do not sell leave the platform.

The model would run at upload time. The seller photographs the item, the classifier predicts a category, and the app pre-fills the dropdown for the seller to confirm or correct. That framing is what makes an 88.65% model shippable. The system is assistive rather than autonomous, so a wrong prediction costs one tap, not a lost sale. A fully automated version at the same accuracy would be miscategorising more than one listing in ten with no one to catch it.

The dominant challenge: distribution shift

The largest risk in this deployment is not accuracy on the test set. It is that the test set does not resemble production. Fashion MNIST images are centred, cropped, evenly lit, grayscale and shot on plain backgrounds. Sellers photograph items crumpled on beds, hanging in dim closets, worn by a person, at angles, in colour, against patterned carpet.

A model that scores 88.65% in the notebook can easily fall well below that on real seller uploads, and the failure would not be visible in any offline metric. Closing that gap requires either a preprocessing pipeline that segments the garment and normalises it to something resembling training conditions, or retraining on actual platform photographs, which is the more durable answer.

Operational concerns

Latency. Inference is not the bottleneck. A forward pass through this network is a few matrix multiplications, single-digit milliseconds on CPU. The real cost sits in image upload over mobile data and in preprocessing. The practical target is returning a prediction before the seller finishes typing the description, so it reads as autofill rather than as a loading state.

Scale. At 109,386 parameters the model is roughly 400 KB, so a single CPU instance handles substantial concurrent traffic. Upload volume is bursty and predictable, concentrated on evenings and weekends, which makes autoscaling on queue depth far cheaper than provisioning for peak. If mobile latency becomes the constraint, the model is small enough to convert to TensorFlow Lite and run on-device, removing the network round trip entirely and working offline.

Integration. This is where deployments usually fail, and it is mostly not a modelling problem. The platform's live taxonomy will have hundreds of categories rather than ten, so an explicit mapping layer is required along with a defined behaviour for items that fall outside the ten known classes. The confidence score has to be exposed rather than just the label, so the interface can show the top three options when the model is uncertain. And every prediction should be logged alongside the seller's final choice, because those corrections are free labelled training data.

Monitoring. The live metric to track is seller correction rate per category, which detects degradation without needing ground-truth labels. Fashion also drifts: new silhouettes appear each season and last year's category distribution stops holding. A rising correction rate in a specific category is the signal to retrain.

The known weakness is already documented. Figure 4 shows exactly where this model breaks, and Shirt, T-shirt, Pullover and Coat are high-volume categories on any clothing marketplace. Shipping this without a confidence threshold on that specific group would put the worst errors in front of the most sellers.

5. Conclusion

A feedforward network with two hidden layers and 109,386 parameters classifies Fashion MNIST at 88.65% accuracy, with macro precision, recall and F1 all within 0.0013 of that figure. It beats the logistic regression baseline by nearly nine points and, more importantly, does so with a two-point train-validation gap instead of sixteen.

The results separate cleanly into two groups. Trouser, Bag, Sandal, Sneaker and Ankle boot are effectively solved above 0.94 F1. T-shirt, Pullover, Coat and Shirt carry roughly 70% of the errors because they share a torso silhouette that survives the loss of detail at 28×28 grayscale. The architecture experiments confirm that this is not a capacity problem: multiplying parameters by five bought 1.4 points, and the widest, deepest dense network still could not break 0.90.

Improvements, ordered by expected return

1. **Use a CNN as the production model.** The unaugmented CNN already reached 0.9109 in 5 epochs, beating the best dense network while training for a third as long. This is the single largest available gain and it is already demonstrated in the notebook.
2. **Add EarlyStopping and ModelCheckpoint.** Figure 3 shows validation loss flat from roughly epoch 5 onward. Monitoring `val_loss` with `patience=3` and `restore_best_weights=True` would cut training time and return the best checkpoint rather than the last one.
3. **Rerun the augmentation test properly.** Train both CNNs for 25 to 30 epochs instead of 5, so the comparison measures generalisation rather than convergence speed. Drop horizontal flip and reduce rotation to 5° to preserve the edge detail that distinguishes the confused classes.
4. **Target the four-class cluster directly.** Options include a two-stage classifier, where a first model routes an image to footwear, bags or upper-body and a specialist model handles the last group, or class weighting that raises the cost of errors inside that block.
5. **Tune the learning rate.** Every experiment used Adam's default of 0.001. A schedule such as `ReduceLROnPlateau` would let the model take smaller steps once validation loss stalls.

Future work

Beyond tuning this architecture, the natural next steps are transfer learning from a pretrained backbone such as MobileNetV2, which brings feature extractors trained on millions of images; higher-resolution input, since 28×28 is the binding constraint on the confused classes and collar or button detail simply is not present at this size; and validating the whole pipeline against real product photographs rather than a curated benchmark, which is the only way to find out what this model actually does in production.

The broader lesson from these experiments is that the choice of model family mattered more than any amount of tuning within one family. Six dense configurations spanning a fivefold parameter range all landed between 0.8556 and 0.8924. A convolutional network cleared all of them in 5 epochs. Matching the model's inductive bias to the structure of the data beats optimising the wrong architecture.

Source notebook: `M_11_A_12_ANN_Neural_Network_Deep_Learning_Basics.ipynb`. Dataset: Fashion MNIST, Zalando Research, 70,000 images across 10 classes. All figures generated from the notebook's own outputs.